

Distributed Algorithms: Multi-Paxos Implementation

Capodanno Mario, Lamperti Luca, Cozzoli Vittorio

Università della Svizzera italiana

Semester: Autumn 2025

Project Overview & Goals

Requirements:

- MultiPaxos implementation for atomic broadcast.
- Multiple concurrent proposers without leader election.
- Three optimizations: 2B to learners, pre-Phase 1, batching.
- Learner catch-up mechanism.

Four Protocol Roles:

Clients Submit values to proposers

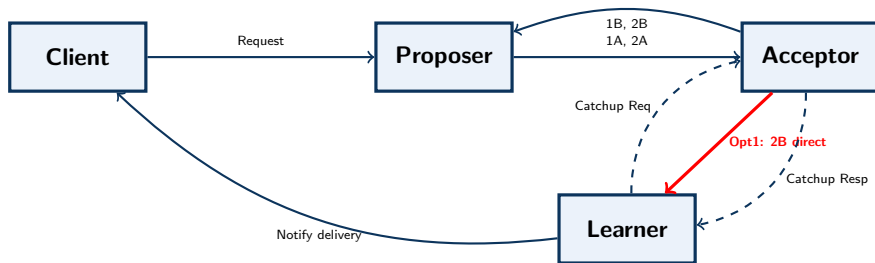
Proposers Init Paxos rounds with batching

Acceptors Run promise/accept logic (Synod algorithm)

Learners Learn and deliver values in total order

System Architecture

Communication Model: IP Multicast (4 groups: clients, proposers, acceptors, learners)



Message Flow:

- Client sends requests to proposers group
- Paxos phases: 1A/1B (prepare/promise), 2A/2B (propose/accept)
- Optimization 1: Acceptors send 2B directly to learners group
- Acceptors and Learners communicate to handle learner catch-up
- Learner notifies client when value is delivered

Atomic Broadcast Requirement

Goal: Decide a *sequence* of values in total order (not just single consensus)

Proposer Strategy:

- Maintains `next_instance` counter
- Runs one Paxos round per instance
- Increments instance after decision
- Code: `proposer.py::run()`,
`proposer.py::run_paxos()`

Learner Strategy:

- Buffers decided instances
- Delivers only in sequential order
- Prints one value per line (for checker)
- Code: `learner.py::deliver()`

Result: learner delivers instance i only after $i - 1$

Safety: Agreement, Integrity, Total Order

Guaranteed under message loss and crash failures:

① Agreement & Validity (Paxos correctness)

- Proposer Phase 1: Collects majority promises, adopts value with the highest `v_rnd`
- Acceptor: Promises only if `rnd_val` $\geq$ `rnd`, accepts only if `rnd_val` $\geq$ `promised`
- Code: `proposer.py::run_paxos()`, `acceptor.py::handle_1A/2A()`

② Integrity (No duplicate values)

- Client assigns unique (`client_id`, `seq_num`)
- Proposer deduplicates via `seen_requests` set
- Learner deduplicates via `delivered_reqs` set

③ Total Order (Sequential delivery)

- Learner uses `buffer` + `next_instance` to enforce order

Multi-Proposer Support (No Leader Election)

Requirement: Support > 1 proposer without leader oracle; handle conflicts via retry

Our Approach

- **Unique Ballot Numbers:** Each proposer uses `{'bal': counter, 'pid': id}`
- **Retry on Failure:**
 - ① Phase 1/2 timeout (no quorum) $\Rightarrow$ increment ballot, retry
 - ② Different value decided (another proposer won) $\Rightarrow$ retry batch at `next_instance`
- Code: `proposer.py::run()`, `utils.py::RndGeq()`

Note: In asynchronous systems:

- **Safety** is always guaranteed (correctness never violated)
- **Liveness** is best-effort (progress depends on network/failures)

Crash Failures & Progress Constraint

Requirement: If majority of acceptors killed $\Rightarrow$ no progress (asynchronous consensus)

Implementation:

- **All state in memory** (no recovery, no stable storage)
 - Acceptor: `instances`, `accepted_values`
 - Learner: `votes`, `buffer`, `catchup_votes`
- **Quorum Check:** Proposer requires $\text{majority} = (\text{num_acceptors} // 2) + 1$
 - If $<$ majority respond $\Rightarrow$ Phase 1/2 timeout
 - No quorum $\Rightarrow$ Paxos "fails" (no decision) $\Rightarrow$ system stalls

Validation: Test03 (one acceptor) correctly stalls (safety maintained)

Implementation Constraints

Two key constraints from project specification:

1. IP Multicast Only

- Four multicast groups (one per role)
- All communication via UDP multicast
- Network layer: `src/utils.py`
 - `mcast_sender()`
 - `mcast_receiver()`

2. No Synchrony

- Cannot use `time.sleep()` in `src/`
- Instead: `select.select()` with timeouts
- Non-blocking socket receives
- Examples:
 - Proposer batching window
 - Client retry mechanism

Note: `select()` allows timeout-based logic without introducing synchrony

Milestone 3

Goal: Reduce latency and minimize Paxos overhead

① Direct 2B to Learners (1 message saved)

- Acceptors multicast 2B to *both* proposers and learners
- Code: `acceptor.py::handle_2A()`
- Critical path: Learners learn without waiting for proposer relay

② Pre-Phase 1 (Pipelining) (2 messages saved)

- When idle (no pending values), proposer pre-runs Phase 1
- Caches promises in `prepared_instance/round/promises`
- Code: `proposer.py::phase1_opt()`
- Benefit: When client values arrive, skip directly to Phase 2

③ Batching

- Proposer collects up to `BATCH_SIZE=50` or `BATCH_TIMEOUT=0.01s`
- Decides multiple values in single Paxos instance
- Code: `proposer.py::collect_batch()`

Learner Catch-up

Requirement: Late joining learner must catch up and learn old accepted values/batches

Three-Step Protocol:

- ① **Learner** sends CATCHUP_REQUEST
 - On startup (`learner.py::run()`)
 - On gap detection: if `received_instance >= next_instance + GAP_THRESHOLD`
- ② **Acceptor** replies with CATCHUP_RESPONSE
 - Sends entire `accepted_values` map (`instance → value`)
 - Code: `acceptor.py::handle_catchup()`
- ③ **Learner** aggregates votes and decides
 - Counts votes per (`instance, value`) pair
 - Once majority reached $\rightarrow$ buffers as decided, triggers `deliver()`
 - Code: `learner.py::handle_catchup()`

Note: Point 3 could be optimized, we will talk about that in the last slide.

Our tests

We added 12 test configurations to check the correctness of the implementation

Test Structure

```
scripts/tests/
├── test01_basic/
│   ├── run.sh
│   └── check.sh
├── ..
├── test04_loss_5/
└── ..
```

Property Verification

Each `check.sh` validates:

Property	Verification Method
Agreement & Order	<code>diff learner1.log learner2.log</code>
Integrity	$\text{Learned} \subseteq \text{Proposed}$
Validity	$\text{Learned} = \text{Proposed}$

Coverage: Basic operation, quorum edge cases, message loss (5-20%), catch-up, multi-proposer contention

Test Results: 12/12 Passed

Test	Scenario	P	A	L	Loss	Catch	Result
test01_basic	Standard operation	2	3	2	0%	No	PASS
test02_two_acc	Minimal quorum (2/2)	2	2	2	0%	No	PASS
test03_one_acc	No majority possible	2	1	2	0%	No	STALL
test04_loss_5	5% packet loss	2	3	2	5%	No	PASS
test05_loss_10	10% packet loss	2	3	2	10%	No	PASS
test06_loss_20	20% packet loss (severe)	2	3	2	20%	No	PASS
test07_catchup	Late-joining learner	1	3	2	0%	Yes	PASS
test08_three_clients	High client load	2	3	2	0%	No	PASS
test09_single_proposer	No contention	1	3	2	0%	No	PASS
test10_four_proposers	High contention	4	3	2	0%	No	PASS
test11_single_learner	Minimal learner	2	3	1	0%	No	PASS
test12_catch_loss	Catch-up under loss	1	3	2	10%	Yes	PASS

Main Difficulties Encountered

1. Deduplication

- Challenge: UDP multicast + client retries $\Rightarrow$ many duplicate messages
- Solution: Track (`client_id`, `seq_num`) at proposer and learner

2. Non Blocking Socket

- Challenge: Cannot use `time.sleep()`, but need timeouts for batching/retries
- Solution: `select.select()` with timeouts,

3. Concurrent Message Handling During Paxos

- Challenge: Client requests arrive during Phase 1/2
- Solution: Buffer incoming client requests in `pending_values` during phases

Future Directions/Improvements

1. Learner.GAP_THRESHOLD Setting (Critical)

- With K total values, if learner has $(K - 3)$ and receives K , won't trigger catch-up (waits for $K + 2$)
- Can violate agreement, since there is the possibility that one learner delivers all K , another stuck at $(K - 3)$
- **Fix:** Set `GAP_THRESHOLD = 1`

2. Catch-up Optimization

- Learner requires the majority of identical `CATCHUP_RESPONSE`
- Since acceptors only store 2B accepted values, a single response is sufficient (by Paxos safety and assuming acceptor state doesn't corrupt)
- Benefit: Faster catch-up from less network traffic